

HarmonyXR GazePose-Context: Multimodal Context Inference for Adaptive XR Training

Lavanya D.

May 14, 2026

Abstract

Adaptive extended reality (XR) systems require more than object interaction; they require an ability to understand whether a user is focused, distracted, transitioning between steps, or inactive. This paper presents HarmonyXR GazePose-Context, a Unity and Meta Quest proof-of-concept that infers user context during a guided XR sorting task using runtime signals already available in a headset-facing implementation. The system uses a three-layer architecture: sensor abstraction, context inference, and XR app shell adaptation. Runtime evidence was collected from a Quest headset run using Unity console logs, filtered QA evidence, processed CSV tables, and a headset video. The evidence run produced 169 QA metric samples and captured all four target context states: Engaged, Distracted, Transitioning, and Idle. The results support the feasibility of a lightweight, interpretable adaptive XR prototype that uses multimodal context signals to drive user-facing guidance. The work is positioned as a headset-validated proof-of-concept; a formal participant study remains future Phase 4 work.

Keywords: extended reality, adaptive XR, context inference, gaze, hand tracking, posture, multimodal interaction, Unity, Meta Quest

1. Introduction

XR training systems increasingly need to support users who shift between focused task execution, attention loss, transitional movement, and inactivity. Traditional XR applications often respond only to explicit controller input or predefined task events. This limits the system’s ability to provide timely help when the user is distracted, paused, or moving between task steps.

HarmonyXR GazePose-Context addresses this problem through a lightweight multimodal context inference prototype. The system observes runtime signals from the XR scene, combines them into a shared signal frame, infers one of four user context states, and exposes adaptive support in the XR app shell. The user task is intentionally simple: sort virtual objects into matching targets. This keeps the research focus on context inference and adaptive response rather than game complexity.

The main contribution of this work is not a new game mechanic or a full user study. It is a working headset-facing proof-of-concept that demonstrates how multimodal XR runtime signals can support context-aware behavior during a controlled training task.

The contributions are:

1. A three-layer implementation structure for adaptive XR context inference.
2. A headset-facing prototype that captures gaze/AOI, spatial context, hand interaction, pinch state, and gaze timing features.
3. A four-state context model covering Engaged, Distracted, Transitioning, and Idle behavior.
4. A user-facing adaptive XR shell that communicates context state and task guidance.
5. A validated evidence package containing a headset video, console logs, filtered state evidence, and processed tables.

2. Related Work

The paper is positioned across four areas of XR research: gaze and attention in immersive environments, multimodal interaction, posture and hand tracking, and context-aware adaptation.

Gaze tracking has been widely studied as an indicator of visual attention in VR and XR. Prior surveys and accuracy studies show that gaze can support object selection, attention monitoring, and adaptive interaction, but gaze alone is often insufficient for robust state inference [1], [3], [11]. Multimodal XR research argues that combining input channels can make interaction more robust than relying on a single modality [4], [8]. This motivates the use of gaze/AOI, hand interaction, spatial context, and posture-related signals in the present prototype.

Body and posture recognition also contribute to user-state understanding. Skeleton-based posture recognition and VR body-tracking research show how body motion can support interpretation of physical behavior in interactive environments [5], [6]. Hand tracking and sensor-fusion approaches are relevant because interaction events and pinch state provide behavioral evidence beyond gaze alone [10].

Context-aware XR systems adapt interface behavior according to task, environment, or user context [7], [9]. However, many systems treat context as externally defined rather than inferred from live behavior. HarmonyXR GazePose-Context contributes a practical, interpretable implementation that links runtime signals to adaptive XR behavior during a concrete task.

3. Research Problem and Scope

The research problem is: how can an XR system infer user context from lightweight runtime signals and use that inference to support adaptive behavior during a training task?

The current paper focuses on prototype implementation and evidence-based validation. It does not claim a completed participant study or statistical comparison. Phase 4 user study work is treated as future work.

The current scope includes:

- Unity and Meta Quest headset execution.
- TrainingSimulation sorting task.
- Four context states: Engaged, Distracted, Transitioning, Idle.
- Runtime QA metrics and context-state evidence.
- Adaptive support and completion feedback inside the XR experience.

The current scope excludes:

- New gameplay systems.
- Phone/mobile support.
- Formal user study statistics.
- NASA-TLX results.
- Baseline-versus-adaptive participant comparison.

4. System Overview

The implementation follows the three-layer structure defined in the implementation guide and WBS: Sensor Abstraction, Context Engine, and XR App Shell. Figure 1 summarizes the runtime pipeline.

Figure 1. HarmonyXR GazePose-Context runtime pipeline.

4.1 Layer 1: Sensor Abstraction

The sensor abstraction layer converts headset and scene signals into runtime fields. Relevant fields include AOI, posture or spatial posture context, boundary mode, fixation duration, dwell ratio, saccade rate, body activity, hand interaction count, pinch state, and distance to task object. These are emitted through QA metrics and logged for validation.

HarmonyXR GazePose-Context Runtime Pipeline

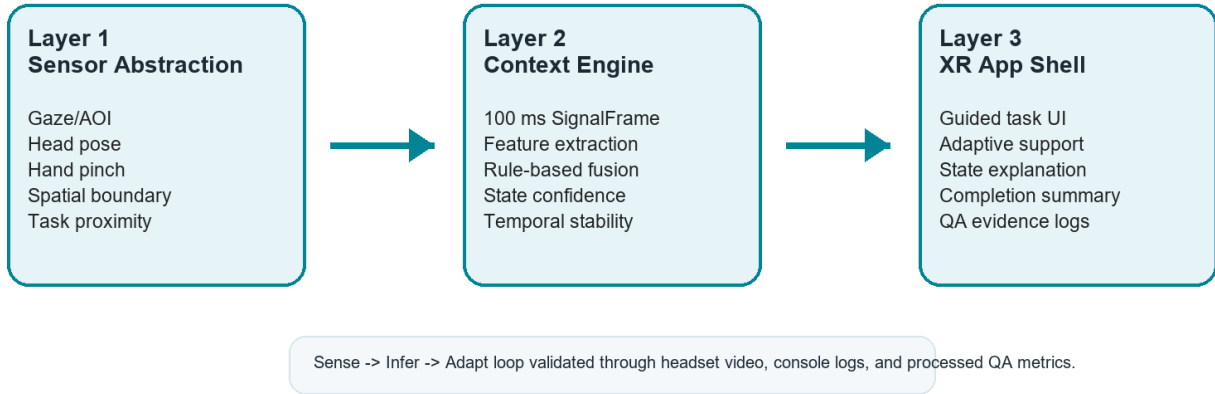


Figure 1: Runtime pipeline

4.2 Layer 2: Context Engine

The context engine consumes the synchronized signal frame and infers a context state. The current implementation uses interpretable rule-based logic and confidence values rather than a black-box model. This design keeps the proof-of-concept explainable for research and stakeholder review.

4.3 Layer 3: XR App Shell

The XR app shell translates inferred state into visible user support. It includes onboarding, task guidance, context-aware feedback, adaptive support actions, QA/evidence logging, and end-of-session completion messaging.

5. Prototype Implementation

The prototype is implemented in Unity 2022.3.62f1 with OpenXR 1.14.3 and Meta XR SDK 201.0.0. The app targets Meta Quest headset execution. The primary scene is TrainingSimulation, where the user sorts a cube, cylinder, and sphere into corresponding pads or receptacles.

The main implementation components are:

- **SignalSynchroniser:** emits runtime signal frames on a 100 ms cadence.
- **ContextDebugger / context pipeline:** evaluates context state and confidence.
- **TrainingSimulationUserGuide:** presents onboarding, task guidance, incorrect-object messaging, and completion summary.
- **AdaptationManager:** maps inferred context states to adaptive XR support and QA metric console logging.
- **ContextLogger:** records context output for analysis.

The proof-of-concept intentionally stays within a narrow research scope. It does not add unrelated gameplay or study-management systems. The goal is to show that adaptive XR behavior can be driven by inferred context signals during a simple task.

6. Context State Model

The system infers four context states.

Engaged indicates that the user is focused on task-relevant content or interacting with the current task. It represents active task participation.

Distracted indicates that attention has shifted away from the task or that task-directed activity is reduced. It supports refocus guidance.

Transitioning indicates movement between task steps, targets, or interaction states. It supports soft guidance while the user changes focus.

Idle indicates low activity, little interaction, or a pause state. It supports rest, recentering, or continuation options.

These states are used as runtime categories for adaptive behavior and evidence reporting. They are not clinical or psychological diagnoses.

7. Evidence Collection Method

Evidence was collected through a combined headset evidence run on May 14, 2026. ADB logcat captured Unity runtime console output while the application ran in the Quest headset. The collected artifacts were organized under **Research_Evidence**.

Primary evidence files include:

- 03_Context_State_Coverage/20260514_run01_full-evidence.mp4
- 05_Stability_Latency/20260514_run01_console-evidence.txt
- 05_Stability_Latency/20260514_run01_filtered-evidence.txt
- 07_Processed_Tables_Charts/qa_metrics_extract.csv
- 07_Processed_Tables_Charts/state_coverage_table.csv
- 07_Processed_Tables_Charts/adaptation_event_table.csv
- 07_Processed_Tables_Charts/stability_latency_table.csv

The evidence run produced 169 QA metric samples. Each QA metric line contained state, confidence, boundary, AOI, posture/spatial context, fixation, dwell, saccade, body activity, hand interaction count, pinch state, and distance to task object.

8. Results

8.1 Context State Coverage

All four required context states appeared in the headset evidence run.

State	Samples	First timestamp	Last timestamp	Found
Engaged	15	05-14 16:19:18.956	05-14 16:21:33.401	Yes
Distracted	30	05-14 16:16:46.104	05-14 16:21:55.453	Yes
Transitioning	100	05-14 16:19:28.044	05-14 16:21:48.434	Yes
Idle	24	05-14 16:16:49.126	05-14 16:22:02.470	Yes

Figure 2. Context state sample counts observed in the headset evidence run.

8.2 Runtime Signal Coverage

The QA metric log confirms that the runtime emitted the key signal fields required for paper evidence.

Signal field	Status	Evidence source
AOI	Present in QA_METRICS	Runtime console log and qa_metrics_extract.csv
POSTURE	Present in QA_METRICS	Runtime console log and qa_metrics_extract.csv
BOUND	Present in QA_METRICS	Runtime console log and qa_metrics_extract.csv

Signal field	Status	Evidence source
FIX	Present in QA_METRICS	Runtime console log and qa_metrics_extract.csv
DWELL	Present in QA_METRICS	Runtime console log and qa_metrics_extract.csv
SACC	Present in QA_METRICS	Runtime console log and qa_metrics_extract.csv
BODY	Present in QA_METRICS	Runtime console log and qa_metrics_extract.csv
HAND	Present in QA_METRICS	Runtime console log and qa_metrics_extract.csv
PINCH	Present in QA_METRICS	Runtime console log and qa_metrics_extract.csv
DIST	Present in QA_METRICS	Runtime console log and qa_metrics_extract.csv

8.3 Adaptive Behavior Evidence

The adaptation evidence table records each trigger state and its expected adaptive response category. Engaged maps to normal task support, Distracted maps to refocus or attention guidance, Transitioning maps to soft movement or task-step support, and Idle maps to pause or adaptive support behavior. The current evidence validates that the trigger states are emitted during headset runtime. Visual confirmation is supported by the full headset evidence video.

8.4 Stability Evidence

The run captured QA metric output from 05-14 16:16:46.104 through 05-14 16:22:02.470. No crash was indicated in the captured evidence, and the QA metric stream continued during the run. Hardware QA latency under 200 ms was reported separately as passed.

9. Discussion

The evidence supports the central claim that a headset-running XR prototype can infer and expose user context using lightweight multimodal signals. The presence of all four context states demonstrates that the model is reachable across the expected behavioral conditions. The signal coverage table demonstrates that the runtime records enough evidence to support analysis of gaze/AOI, posture or spatial posture context, hand interaction, pinch state, and gaze-derived timing features.

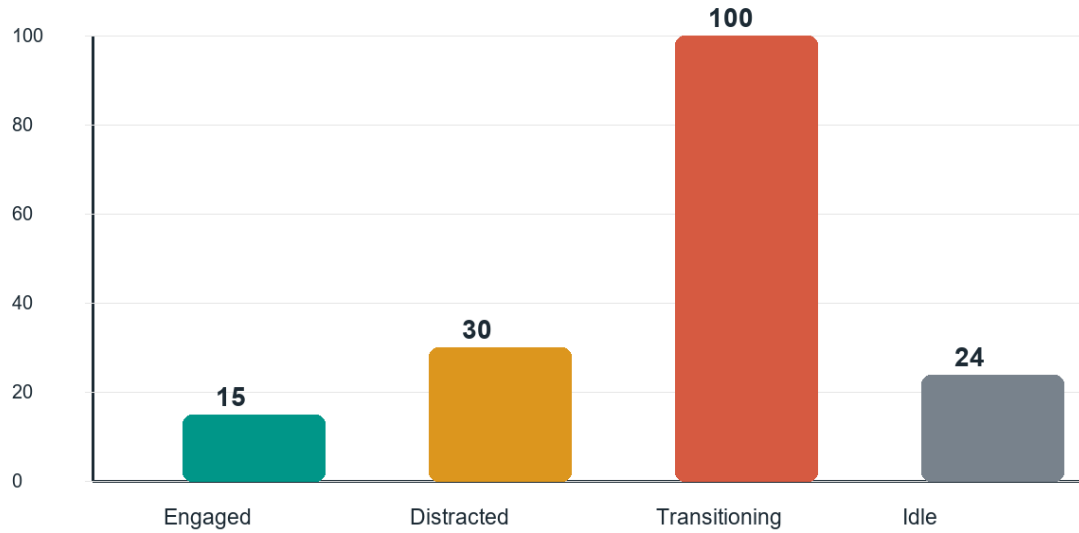
The communication improvements in the app shell are also important. Earlier versions of the prototype risked appearing like a technical debug demo. The current presentation better explains the research purpose, active task, inferred state, and adaptive support. This matters because the proof-of-concept is intended for first-time users, clients, and research stakeholders, not only developers.

10. Limitations

This work remains a prototype-level validation rather than a completed user study. The evidence comes from a headset evidence run, not from a multi-participant controlled experiment. The results should therefore be interpreted as proof-of-concept validation, not as statistical evidence that adaptive XR improves task performance or workload.

Several implementation boundaries should be noted. The `room_scale` posture value should be interpreted as spatial or boundary context, not strict sitting/standing recognition. Sitting or standing should only be discussed where explicit sitting or standing values appear in the logs. The current context log format is JSONL-style runtime evidence rather than a single JSON array document. Some guide-level QA requirements, such as formal participant self-report ground truth and baseline/adaptive comparisons, remain Phase 4 work.

Context State Coverage in Headset Evidence Run



Source: Research_Evidence/07_Processed_Tables_Charts/state_coverage_table.csv

Figure 2: State coverage

11. Future Work

Future Phase 4 work should conduct a formal within-subjects study comparing baseline and adaptive conditions. The planned study should collect task completion time, sorting errors, context detection accuracy against ground truth, and subjective workload. Additional future work may include richer locomotion modeling, face tracking, EEG or physiological signals, multi-user context modeling, and personalization of inference thresholds.

12. Conclusion

HarmonyXR GazePose-Context demonstrates a working adaptive XR proof-of-concept that infers user context from runtime signals and uses that context to support task guidance. The headset evidence package confirms all four target states and 169 QA metric samples containing key signal fields. While formal Phase 4 participant evaluation remains future work, the current implementation provides a credible systems prototype and evidence base for an adaptive XR research paper.

Acknowledgment and Disclosure

This draft was prepared from the current OpenSpatialAI implementation, HarmonyXR planning documents, and the May 14, 2026 evidence package. AI-assisted development and documentation tools were used during project work; final submission should include the disclosure wording required by the target venue.

References

- [1] Moreno-Arjonilla et al., “Eye-tracking on Virtual Reality: A Survey,” *Virtual Reality*, 2024.
- [2] Long et al., “Multimodal Detection of External and Internal Attention in Virtual Reality using EEG and Eye Tracking,” *MuC*, 2024.

- [3] “Comparison of Gaze Accuracy and Precision in Real-World and Virtual Reality,” *Virtual Reality Journal*, 2020.
- [4] Rakkolainen et al., “Technologies for Multimodal Interaction in Extended Reality—A Scoping Review,” *Multimodal Technologies and Interaction*, 2021.
- [5] Bustamante et al., “Skeleton-Based Posture Recognition for Home Care,” *Expert Systems*, 2025.
- [6] Neidhardt et al., “VR-based Body Tracking to Stimulate Musculoskeletal Training,” *arXiv*, 2023.
- [7] Davari and Bowman, “Towards Context-Aware Adaptation in Extended Reality,” *arXiv*, 2024.
- [8] Kim et al., “From Controllers to Multimodal Input: XR Interaction Evolution,” *Sensors*, 2026.
- [9] Ramiotis and Mania, “CONTEXT-GAD: Context-Aware Gaze Adaptive Dwell Model,” *VRST*, 2025.
- [10] Lei et al., “A Novel Sensor Fusion Approach for Precise Hand Tracking in VR,” *Biomimetics*, 2023.
- [11] Selaskowski et al., “Gaze-based Attention Refocusing Training in VR for ADHD,” *BMC Psychiatry*, 2023.